Why the code is shaped this way

The viva asks "why". Each rule below has one reason you can say in a sentence. How it works (the files, step by step) is in `docs/HOW-IT-WORKS.md`; this file is the reasons.

What the rebuild did

Every page keeps its look from the first version. Underneath, the same pages now take far less code:

	First version	Now	
Page + component PHP (views)	15,551 lines	7,822	–50 %
CSS	17,235	12,300	–29 %
JavaScript	18,918	3,104	–84 %
Example data instead of a database	most pages	6 files left (`app/Mock/`)	your tasks remove them

(Non-blank lines, counted 2026-11-01.) How:

- **Copies became components.** The same card typed in five pages is now one component with a `$variant`.
- **JavaScript that only drew HTML became PHP.** The server builds the page; JS is left for what must happen in the browser (dropdowns, pop-ups, the calendar).
- **The browser does it for free.** `<dialog>` for pop-ups (Esc, the dark backdrop, focus), `required` for empty fields, `<form method="get">` for filters — instead of hundreds of lines of JS.
- **One look per kind of thing.** One button (`c-btn` + a look), one field (`c-field__input`), one card, one banner, one set of colours, sizes, corners and shadows (`global.css` tokens). `bin/check.sh` refuses anything else.
- **Real data, one table per thing,** with every rule in one place.

The structure

Rule	Why
Views in per-role folders (`app/Views/student/…`)	Each role's pages are easy to find; a role sees only its own pages
A page only lists components (`$styles`, `$scripts`, `$grids`)	Pages stay ~30 lines; a page loads only the CSS / JS it uses
One component = its PHP + own CSS + own JS	Change a component in one place; nothing else breaks
`$variant` instead of a copied file	One fix fixes every page that uses it
Page rows in `css/layout/*-grid.css`	Arranging (columns, gaps) is kept apart from how a piece looks
`global.css` = tokens, resets, the app shell only	One name per colour: a colour change is one line
No frameworks, no libraries (only Dompdf, for the PDF)	We can explain every line; nothing hidden

MVC on the server

Rule	Why
Models are the only place with SQL	One place to check for injection; controllers stay readable

Rule	Why
Controllers decide, views only show	Business rules are in one place, not scattered through HTML
Services (`app/Services/`) for work that is neither SQL nor a page	e.g. `TermResults` works out averages and positions; `JoinRules` decides who may join
Totals, averages, positions are worked out live, never stored	A corrected mark changes every number at once; nothing goes stale
Assignments per school year (`class_students` with a `year`)	Last year's classes, teachers and marks stay true after year-end
Accounts and records are never deleted (switched off, archived, retired)	History and old reports keep working

Security

Rule	Why (the attack it stops)
Prepared statements only; column names only from a fixed list	SQL injection
`e()` on every printed value	XSS: a script typed into a name would run in other people's browsers
CSRF token on every POST, checked once in `core/Router.php`	Another website can't send our forms with a signed-in person's session
Logout is a POST	A link or image on another site can't sign people out
`postText()` / `postPassword()` / `postList()` only	`field[]=x` or other odd values can't crash a page or slip through
`requireRole()` + `can()` on every action	Typing another role's address does nothing
The same message for an unknown and a real account	Strangers can't find out which IDs exist
A wait between emails / codes; tries counted; a pause after wrong passwords	Inbox flooding and password guessing
Passwords with `password_hash()` ; codes stored hashed	A stolen database doesn't give away passwords
`transaction()` for changes that belong together	Never half-saved (e.g. a team and its whole roster)
JSON requests get JSON on every path (`refuse()`)	The page's script always gets an answer it understands
Secrets only in `.env` (not in git)	Keys can't leak through the code
Errors show `errors/error.php` ; details go to `storage/logs/`	People never see code or file paths

Rules in one place

Rule	Why
Who may do what → `config/permissions.php` (`can('…')`)	Changing who may do something is one line, and you can show the examiner the whole list
School rules with numbers → `config/white_label/rules.php` (`rule('…')`)	"Pass mark 40 instead of 35" is one line. **The code check will ask this kind of thing**
A feature's own options → `config/{feature}.php`	Limits, lists, sizes for one feature sit together

White-label (another school, no code changes)

Need	How	Why
Name, logo, contacts	`school('name')`	Another school changes `config/white_label/school.php` only
Grade / Class / Term / Student	`word('class')`, `plural()`, `words('{Class} {Teacher}')`	Another school may say "Form" or "Semester"
Switch a module off	`feature('notices')`	Its pages, routes, menu link and dashboard cards all go
Dates	`country('date_format')`, `<html lang>` in JS	Each country writes dates its own way

How we keep it clean

Tool	Does
`bin/check.sh`	Syntax, banned things (`!important`, inline styles…), dead code, duplicates, typed school words — 0 errors before every merge
The edit hook (`.claude/hooks/check-edit.sh`)	Warns Claude straight after an edit that breaks a rule
`rules-reviewer`	Reviews a branch against `CLAUDE.md` before a pull request
`/next-feature`	One complete plan before any code
`/explain`	Walks you through a file until you can explain it
`docs/CHECKLIST.md`	Every decision, with its date and the reason
The GitHub check	Runs `bin/check.sh` on every pull request